

Susceptibility Model — Training & Validation Report

Generated 2026-09-02. Scope, stated plainly and unchanged: **road-corridor, zone-level landslide susceptibility for the Sikkim pilot. Not event-time prediction.**

Dataset

774 positive + 774 negative = 1548 rows, data/processed/training_dataset.csv (post-deduplication, see docs/duplicate_and_bias_audit.md). Dataset construction was not touched — no bug found that would justify it.

Validation strategy — spatially defensible, not a naive random split

Why not random: this session's own audits showed strong small-scale spatial clustering — positives sit a median ~7-10m from a road, and the road-corridor sampling design means nearby points often share nearly identical terrain. A random point split would very likely place near-duplicate neighbors in both train and test, inflating apparent performance without the model learning anything transferable.

How it was built: 1. Every point assigned to a 2km × 2km grid cell in UTM (metric, not degrees) — scripts/ml/spatial_cv.py. 2km was chosen as an order of magnitude larger than the 200m positive/negative exclusion buffer already established in sampling, and larger than the scale terrain features typically vary over in this terrain. 2. Whole cells (485 of them) randomly assigned to 5 folds — never split a cell's points across folds. 3. **Measured, not assumed:** raw grid blocking alone left train/test boundary points only 56-69m apart in places — too close. A 200m buffer was then applied per fold, dropping train points within 200m of any test point (2-6 points per fold, out of ~1200+ remaining). After buffering, every fold's actual measured train-test separation is ≥**208.9m** (verified per fold, not asserted).

Train/test counts per fold (test fold size shown; train = remaining ~1200 points minus buffer drops): | Fold | Test points (pos/neg) | Train points after buffering | Min train-test distance | |---|---|---|---| | 0 | 341 (168/173) | 1201 | 226.3m | | 1 | 260 (126/134) | 1286 | 220.2m | | 2 | 336 (162/174) | 1206 | 208.9m | | 3 | 331 (177/154) | 1211 | 226.3m | | 4 | 280 (141/139) | 1262 | 208.9m |

Spatial overlap: none, by construction, above the 200m buffer threshold — verified per fold above.

Every point is predicted exactly once, by a model that never saw it (or anything within 200m of it) during training. Out-of-fold predictions from all 5 folds are pooled into one set covering the full dataset for headline metrics; per-fold metrics are also kept separately to assess stability.

A **naive random 80/20 split** was also run per experiment, purely as a diagnostic contrast — never used for model selection — to quantify how much a naive approach would have overestimated performance.

Full metrics table (spatial CV, pooled out-of-fold, threshold=0.5)

Experiment	ROC-AUC	PR-AUC	Precision	Recall	F1	Fold AUC std	Naive-split gap
Logistic Regression — baseline	0.674	0.618	0.640	0.630	0.635	0.019	+0.030

Logistic Regression — extended	0.714	0.701	0.661	0.602	0.630	0.031	+0.037
Random Forest — baseline	0.697	0.671	0.630	0.690	0.659	0.018	+0.015
Random Forest — extended	0.735	0.733	0.677	0.694	0.685	0.020	+0.017

Confusion matrices (rows=actual [neg,pos], cols=predicted [neg,pos], pooled out-of-fold): - LR baseline:

[[499, 275], [286, 488]] - LR extended: [[535, 239], [308, 466]] - RF baseline: [[461, 313], [240, 534]] - RF extended: [[518, 256], [237, 537]]

ROC and Precision-Recall curves for all 4 experiments: [data/models/roc_pr_curves.png](#). Feature importance (RF) / standardized coefficients (LR) for all 4: [data/models/feature_importance.png](#).

Diagnostic checks

- 1. Suspiciously high performance?** No. All four experiments land in 0.67-0.74 ROC-AUC — a modest, believable range for 4-5 features on field-survey data with known label ambiguity. Nothing here resembles the >0.95 AUC that would demand real suspicion.
- 2. Spatial leakage?** Checked directly by comparing the naive random split against the spatial CV for every experiment. The gap is small (+0.015 to +0.037 AUC points) and consistent — real but modest inflation, exactly what buffered spatial CV is supposed to correct for. Nothing catastrophic was hiding.
- 3. Duplicate/overlap issues?** Zero duplicate coordinates in the training data (resolved before this run — see the dedup audit). Zero positive/negative coordinate overlap (carried over from dataset construction).
- 4. Class separation / overfitting.** Logistic Regression's train-set and spatial-CV AUCs are nearly identical (0.680 vs 0.674 baseline; 0.722 vs 0.714 extended) — a linear model with little room to overfit 4-11 features. **Random Forest shows a real gap** (0.896 train vs 0.697 CV baseline; 0.866 vs 0.735 extended) — it fits training-specific structure that doesn't fully generalize. This is normal RF behavior, not a red flag on its own, but it's the main reason RF's advantage should be trusted at its *spatial-CV* number (0.735), not anywhere near its in-sample number.
- 5. Does land_cover_class materially improve spatial generalization?** Yes, consistently: +0.040 AUC for Logistic Regression, +0.038 for Random Forest — the same-direction, similar-magnitude improvement across two very different model types is meaningful, not noise. This is consistent with (not proof beyond) the land-cover bias audit's finding that built-up land carries signal beyond simple road-proximity confounding.

Model selection

Ranked by held-out spatial ROC-AUC, then fold-to-fold stability (std) as tiebreaker — not a single metric, and not the naive-split number:

- 1. random_forest__extended** — ROC-AUC 0.735, stability std 0.020
- logistic_regression__extended — ROC-AUC 0.714, stability std 0.031
- random_forest__baseline — ROC-AUC 0.697, stability std 0.018
- logistic_regression__baseline — ROC-AUC 0.674, stability std 0.019

Best model

Best model	Random Forest	
Features	Interception, slope, curvature, distance_to_drainage, land_cover_class (7 one-hot columns)	
RCC	0.75	AUC
PR	0.73	AUC
Precision	0.67	Recall
Recall	0.64	
F1	0.685	

Why highest
selected out
spatial
ROC-AUC
and
PR-AUC
of
all
4
experiments,
with
fold-to-fold
stability
(std
0.020)
comparable
to
the
more
conservative
baseline
models
—
not
an
outlier
that
happens
to
win
once.
Land-cover's
contribution
here
is
corroborated
by
the
separate
bias
audit,
not
just
this
run's
numbers.

Main limitations

(1)
Notable
gap
(0.866
vs
0.735)
vs.
Logistic
Regression's
near-zero
gap
—
more
prone
to
overfitting,
mitigated
but
not
eliminated
by
max_depth=8,
min_samples_leaf=5.
(2)
The
built-up
land-cover
signal's
causal
mechanism
(genuine
destabilization
vs.
GSI
documentation
bias)
remains
unresolved.
(3)
Positive
samples
are
still
concentrated
near
roads
—
this
model
has
not
been
evaluated
on
terrain
far
from
any
road.
(4)
ROC-AUC
0.735
is
a
real
but
modest
result
—
good
enough
to
rank
zones
by
relative
risk,
not
precise
enough
to
be
treated
as
a
probability
of
failure.

Artifacts saved

File	Contents
data/models/susceptibility_model.joblib	Fitted Pipeline (StandardScaler + RandomForestClassifier), the exact feature column list, and model_version — bundled together so preprocessing can never drift from the model that expects it
data/models/validation_report.json	Full machine-readable metrics, spatial-CV config, risk-tier thresholds, all 4 experiments' summaries, limitations
data/models/roc_pr_curves.png	ROC + PR curves, all 4 experiments
data/models/feature_importance.png	RF feature importances / LR coefficients, all 4 experiments
data/models/demo_point_predictions.csv	See below

Model version: random_forest-extended-v1-20260902

Risk tier thresholds: tertiles of the final model's own predicted-probability distribution on the training data — low < 0.423 <= moderate < 0.595 <= high. This is the team's own choice, not literature-sourced, same honesty pattern as the rainfall-alert multiplier table.

"Predictions for all pilot zones" — honest gap

No real Sikkim pilot zone polygons exist in the database yet. scripts/seed_zone.py has only ever been run against the documented Gulf-of-Guinea placeholder — there is nothing real to predict on. Producing "zone predictions" against a fake boundary would be actively misleading, so this wasn't done.

What was done instead: scripts/ml/predict_zone_susceptibility.py runs the complete real pipeline (DEM + land-cover extraction → trained model → score/tier) on the 1548 training-data points themselves, as a readiness demonstration — clearly labeled as such in the script and its output, never as real zone predictions. Sanity check passed: mean predicted score for actual positives (0.617) is meaningfully higher than for actual negatives (0.385) — the model discriminates in the expected direction. Risk tiers split close to evenly (511 low / 526 moderate / 511 high), as expected from tertile construction.

The moment Data/GIS lead's real boundary exists, predict_at_points() in that same script is the exact function to call — no new code needed.

API contract verification — done for real, not just checked on paper

1. Seeded the existing placeholder test zone (config/pilot_zone.example.geojson, the obvious Gulf-of-Guinea fake) into the live Postgres database
2. Took a real prediction from the trained model (susceptibility_score=0.7311, risk_tier="high", model_version="random_forest-extended-v1-20260902")
3. Issued a real PUT /zones/{id}/susceptibility HTTP request against the running backend — **200 OK**
4. Confirmed via GET /zones that the value persisted correctly in the database

The model's output shape matches the existing SusceptibilityUpdate schema exactly — no backend changes were needed. (The test zone remains in the local database, clearly named "TEST ZONE - not real Sikkim boundary" — flagging this rather than silently leaving it.)

Stopping here, as instructed. Dashboard and alert system untouched. No claim of event-time prediction anywhere in this pipeline or its outputs.